

FEBRIC Onboarding & Feature Implementation Research

Executive Summary

FEBRIC's mobile app requires robust yet user-friendly account management, secure API key handling, and an AI-driven design workflow on responsive layouts. For **onboarding**, we recommend passwordless login via email OTP and Google Sign-In. Email OTP entails sending a 4–8 digit code (short-lived, e.g. 5–15min) to the user's email, which they enter to authenticate [54†L111-L116] . This provides high security (cryptographically-generated, single-use) and “intent confirmation” (users consciously retrieve and enter the code) [54†L118-L122] . Google Sign-In uses OAuth2: the app receives an ID token from Google which the backend must verify (JWT signature, issuer, audience, expiry) [17†L276-L284] . After verification, the `sub` (user's Google Account ID) is used as the user ID [17†L252-L260] . A sequence diagram for these flows is shown below.

For **API key management**, the user will **bring-your-own key (BYOK)** for Gemini. In practice, users must create a Google Cloud API key (restricted to the Gemini API per Google's advice [25†L139-L142]) and enter it in the app. The app securely stores the key (e.g. using `flutter_secure_storage` [22†L75-L83]) and periodically validates it by making a benign AI request (e.g. listing models). The UI should have clear states: *No key* (red banner “Add API Key”), *Key added and valid* (green status), *Key invalid/expired* (red error). Table **UI States** below summarizes these.

With an API key present, AI features (chat, motif generation) become available. Otherwise, such features should be visually disabled but not block the entire UI. Smashing Magazine warns that disabled controls without explanation confuse users [29†L152-L159] . Instead of hidden features, we recommend showing a non-blocking banner or tooltip (e.g. “AI features are disabled – add an API key to unlock them.”). Always accompany disabled UI with explanatory copy. Use high-contrast, accessible labels (per accessibility best practices [3†L113-L117]) and ensure any disabled buttons have ARIA hints.

Asset import will use Flutter's native file pickers. On Android, the [Storage Access Framework \(SAF\)](#) (via `ACTION_OPEN_DOCUMENT` intents) lets users select files from any provider [31†L489-L498] . In Flutter, the `file_picker` plugin is recommended [32†L88-L97] : it works on Android/iOS/Web/Desktop, supports filtering by extension, and returns file metadata/stream. For large files (multi-MB images or PDFs), use `file.readStream` instead of loading into memory [32†L119-L127] . Most pickers use system dialogs, so no extra storage permissions are needed in typical cases [32†L140-L148] . Supported formats should include PNG/JPEG/TIFF (for raster images), SVG (vector), and a custom “.febric” JSON project file. The UI will provide buttons (gallery, camera, file) on the chat input bar. When a user attaches an image, the `ConversationItem` model will have a new type, e.g.:

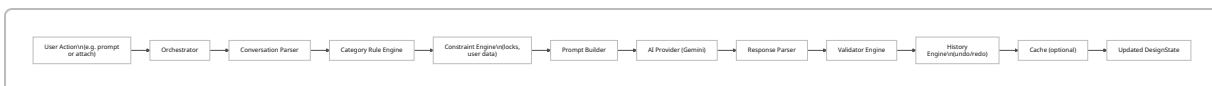
```
class ConversationItem {
    String type; // "text" or "image" or "file"
    String content; // message text or file path/URL
    Map<String, dynamic>? metadata; // e.g. image dimensions, mime-type
    DateTime timestamp;
}
```

We also define an **AIRequest** and **AIResponse** schema. For example:

```
// AIRequest
{
    "image": "<current canvas image bytes or URI>",
    "masks": { /* MaskSet of locked areas */ },
    "conversation": [ /* previous ConversationItems */ ],
    "userPrompt": "Replace the border with paisley.",
    "constraints": { "keepMotif": true, "lockedLayers": ["PrimaryMotif"] },
    "model": "gemini-2.5-pro",
    "params": { "steps": 50, "cfg": 7.5 }
}
```

```
// AIResponse
{
    "image": "<generated image bytes or URI>",
    "promptUsed": "...final prompt text...",
    "provider": "Gemini",
    "model": "gemini-2.5-pro",
    "executionTimeMs": 1234,
    "warnings": [],
    "errors": []
}
```

The **AI Orchestrator** pipeline will sequence these steps: parse the conversation + attachments, apply category rules, merge with layer locks, construct the final prompt, call the AI provider, parse the response, validate it, and update the design state. The diagram below illustrates this flow:

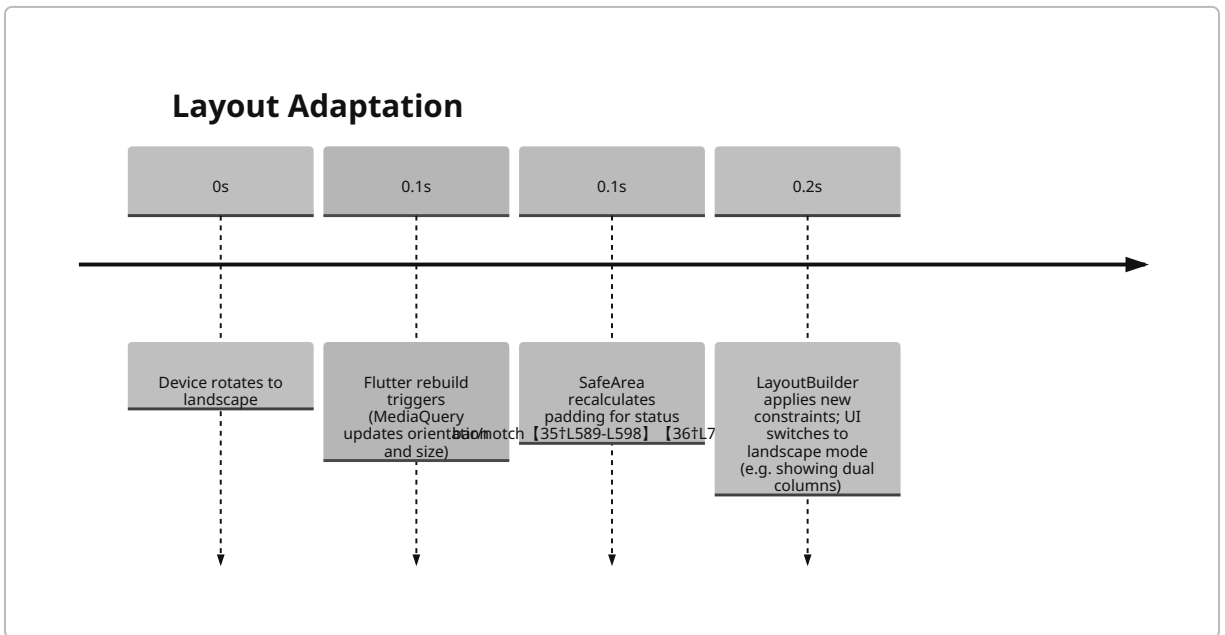


For **constraints and prompt building**, any uploaded asset or user command generates structured constraints. For example, if a user uploads a motif image, the Constraint Engine may set `"targetLayer": "HeroMotif"` or add "use this motif" to the prompt. If the user says "keep colors the same," we set a color-lock constraint. The final prompt is layered: a system prompt (design assistant role), category rules (e.g. "design a Pakistani kurta print"), followed by prior conversation for context, then the

user's instruction, and finally constraint instructions like *"Do not modify the primary motif or base colors."* This ensures the AI knows exactly what to do. For example:

System: You are an AI assistant that designs textile prints.
 Rule: The design is for a Pakistani kurta (long tunic). Include components like yoke, sleeves, etc.
 Conversation: [previous user asks 'Add more floral elements']
 User: Replace only the border with paisley motifs, keep everything else the same.
 Constraint: Do NOT change the existing motifs or base colors.

When it comes to **responsive layouts and SafeArea handling**, Flutter's `SafeArea` and `MediaQuery` tools are key [35†L589-L598] [36†L75-L82]. Always wrap the main content (e.g. the `Scaffold` body) in a `SafeArea` to avoid status bars, notches, and camera cutouts [35†L589-L598]. `SafeArea` uses `MediaQuery.padding` (derived from `viewPadding` and `viewInsets`) to apply appropriate insets automatically [36†L75-L82]. For landscape mode, ensure the UI adapts: you may switch to a side-by-side design (e.g. chat on left, canvas on right). Use `MediaQuery.of(context).orientation` or `LayoutBuilder` to detect size classes (e.g. `width>600dp`) and switch to a two-column layout on tablets/landscape. The timeline below shows how Flutter responds to an orientation change:



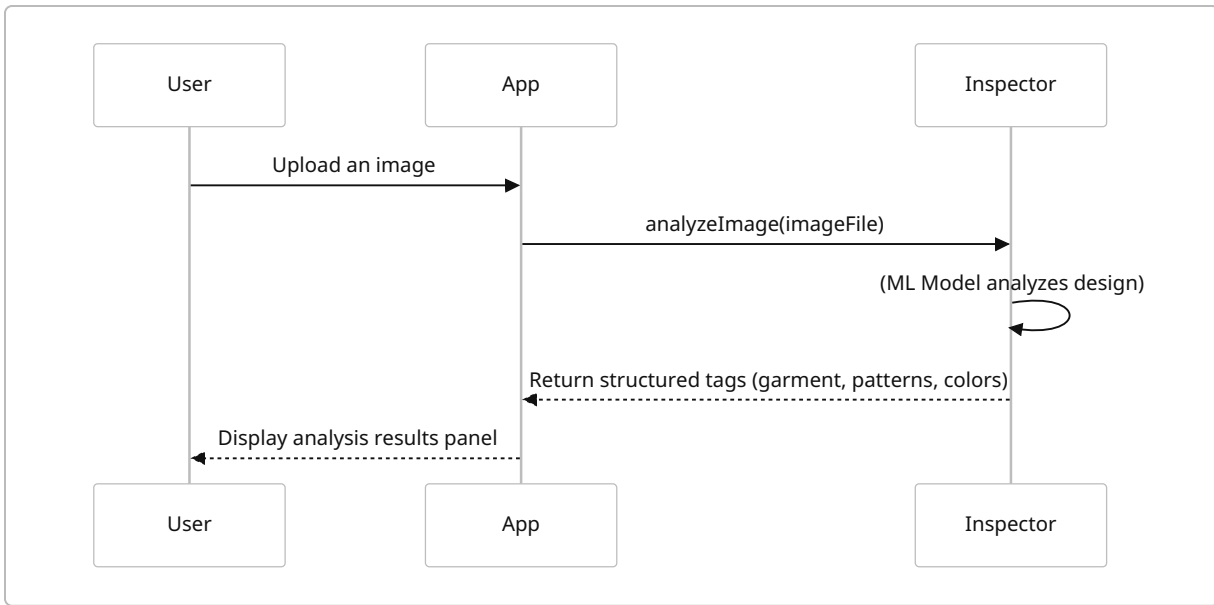
The **Design Tree** will be a nested model: a Project contains one or more Textile Artworks (design files), each with Sections (e.g. Dupatta, Kurta, Pant, Border). Each Section has Layers (e.g. for Dupatta: Top Pallu, Middle Field, Bottom Pallu, Side Borders, Corner Motifs). Each layer has editable properties and state (for example: `locked:bool`, `visible:bool`, `enabled:bool`, `lastModified:timestamp`). The UI will present these in a collapsible tree panel. Each layer row shows an “eye” icon (visibility toggle), a “lock” icon, and possibly color swatches or thumbnails. Users can drag layer handles on the canvas to resize sections. Repeat-pattern layers (e.g. All-over print on Kurta) will have controls for pattern type and scale.

Section/Layer	Layer Types	Operations
Base Fabric	Base Color, Fabric Texture	Change base color, import fabric texture.
Hero Motif	Primary design element	Import motif, change color, rotate/scale.
Secondary Motif	Supporting motifs	Import motif, adjust placement, colorize.
Filler Motif	Small filler patterns	Add/remove, change color.
Outline/Hem	Borders, edge trims	Import border graphic, adjust width, recolor.
Sleeve	Sleeve pattern/shape	Modify length, import pattern, recolor.
Neck/Collar	Neck shape, design	Import motif/pattern, change color.
Shadow	Drop shadow effect	Adjust opacity/intensity.
Texture	Overlay textures (e.g. linen)	Import texture, adjust blend and intensity.

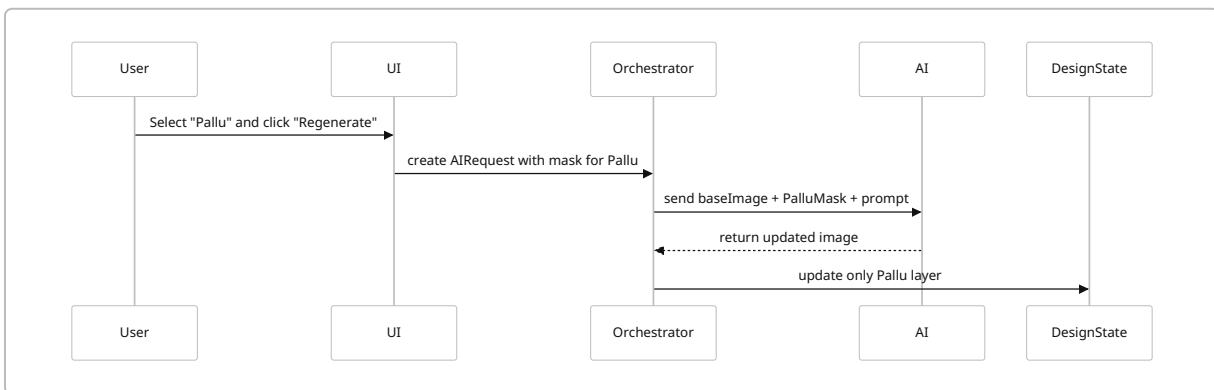
For instance, if the user wants to repeat a motif across the Kurta, they could add a “repeat-pattern” on the Hero Motif layer (with options like grid, mirror, half-drop, etc.). All interactions (drag to resize a Pallu, sliders to adjust repeat spacing, color pickers, etc.) should be intuitive and real-time.

The **AI Assistant** features include:

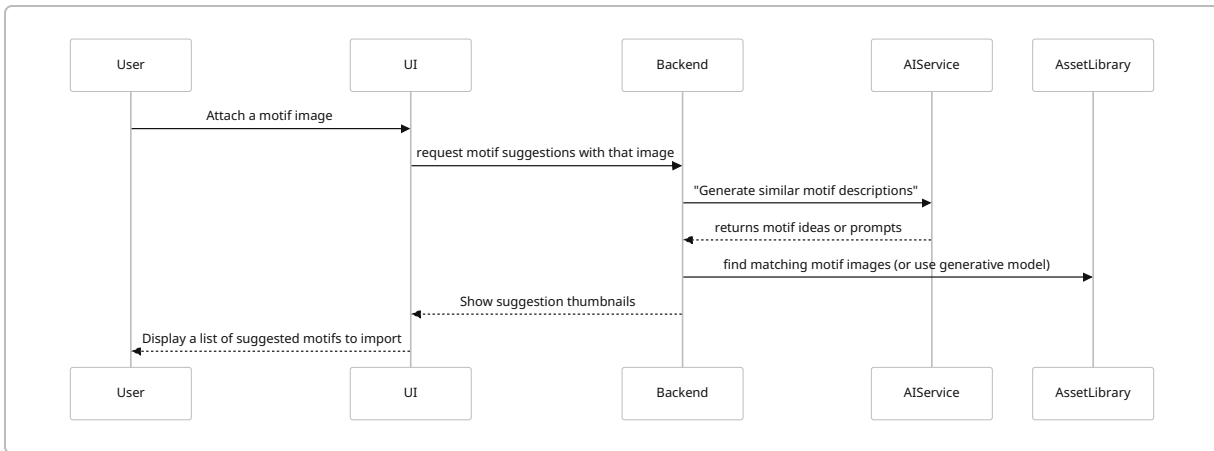
- **Smart Textile Inspector:** When the user uploads a design image or motif, this service analyzes it. For example, it can identify garment type (“Pakistani Kurta”), fabric (“Cotton Lawn”), and visual style (“floral paisley pattern”). This likely uses an AI vision model or API (e.g. Gemini Vision or a custom CNN) to classify or tag the image. A sequence example:



- **Partial Regeneration:** If the user locks certain layers (e.g. the motif) and asks to change only another section (e.g. "Change only the pallu pattern"), the app will mask out locked areas and send only the target area to the AI for inpainting. For example, a sequence:

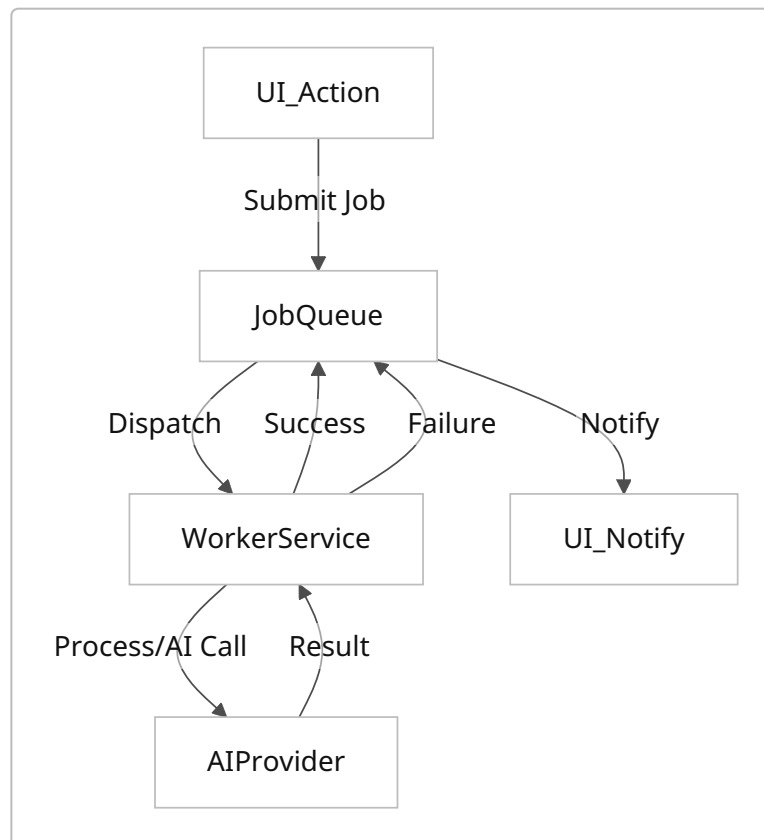


- **Inspired Motif Suggestions:** After uploading or selecting a motif, the user can request similar suggestions. The app would send a prompt to the AI like "List 5 trending motifs similar to this paisley design," then retrieve or generate small preview images.



These AI features require backend services (or app logic) for image analysis, constraint parsing, prompt engineering, and validation. For example, **Validation rules** might check that if a layer is locked, the AI's output did not alter it; or that design rules (like a kurta must have sleeves) are not violated. If invalid, the UI can alert the user and ignore the change.

For long-running tasks (e.g. generating a high-res image), use a **job queue and background worker**. On mobile, one could use a background isolate or WorkManager to prevent blocking the UI. On the server, a message queue (RabbitMQ, AWS SQS, etc.) can handle tasks with retry and priority. A simple flow:



The app should support canceling a job (remove from queue), retry with backoff on transient errors, and limiting concurrency (e.g. one AI call at a time to manage memory). Progress can be reported via a progress bar or spinner with status ("Loading, 30%").

For **theming and premium UI**: FEBRIC needs a polished, modern look. We should implement multiple themes via Flutter's `ThemeData`. Provide at least Light and Dark themes (following Material guidance). For AMOLED screens, a "true black" dark theme can be offered (saving battery and looking sleek). Accent colors should be user-selectable (let users pick a primary color from a palette or custom color-picker). Use a clean icon set (e.g. Material or custom flat 2D icons) and a professional font (like Montserrat or a refined serif for headings). Visual language can include subtle shadows, rounded cards, and glassmorphism panels. Since we aim for 120fps animations on new devices, optimize for performance: target <8ms per frame [\[48†L900-L905\]](#), minimize expensive operations (avoid unnecessary `Opacity` widgets – use `AnimatedOpacity` or `FadeInImage` instead [\[48†L924-L932\]](#)), and pre-cache complex layers. Use Flutter's GPU-accelerated transforms and limit overdraw to keep animations smooth.

Security and privacy are paramount. Store the user's API key and any sensitive tokens in secure storage (Android Keystore/iOS Keychain via `flutter_secure_storage` [\[22†L75-L83\]](#)). User account info (email) should also be stored securely or on a backend. If any design images or project files are stored, treat them carefully: if stored locally, consider encrypting (or marking for deletion on logout). If stored on a server (cloud sync), use encryption at rest and in transit. Respect GDPR: user-uploaded images could contain personal data, so obtain consent and allow users to delete them. Provide a clear privacy policy. For cached data (like AI responses or analysis), use a time-to-live (e.g. purge after 24–72 hours) and limit cache size (we suggest ~100MB or 500 entries as a cap). For example, the caching design mentioned a 500-entry limit with expiration for each entry. Similarly, undo/redox history should be bounded (e.g. 200 steps max) to prevent unbounded memory use.

Deliverables and Next Steps: Below we summarize key fixes and provide diagrams and prompts to guide implementation.

- **Prioritized Fix List (Critical Issues):**
- **Landscape Safe-Area** – Ensure all screens (especially "Design Tree" and "Canvas") wrap content in `SafeArea` so status bars/notches do not overlap UI. [\[35†L589-L598\]](#).
- **Asset Import UI** – Add a file picker button on the chat input bar (with gallery/camera options). Handle file types (PNG, JPEG, SVG, .febric) via `file_picker` [\[32†L88-L97\]](#). Show upload progress and errors.
- **AI Chat Attachments** – Allow attaching images in chat; update `ConversationItem` model to include images. Display attached images inline with chat bubbles.
- **Design Tree Population** – Expand the tree to include sections (Dupatta, Kurta, Pant, Border) and sublayers as per design. Add lock/visibility toggles. Ensure interactions (e.g. tapping a layer zooms to that section) are implemented.
- **API Key Banner** – In the top bar or settings, show a banner if no API key is present (e.g. "Add API Key to enable AI features" in red). When a key is entered, validate it and update the banner/status (green if valid, red if invalid).
- **Folder Structure Suggestion:** Organize code by feature:

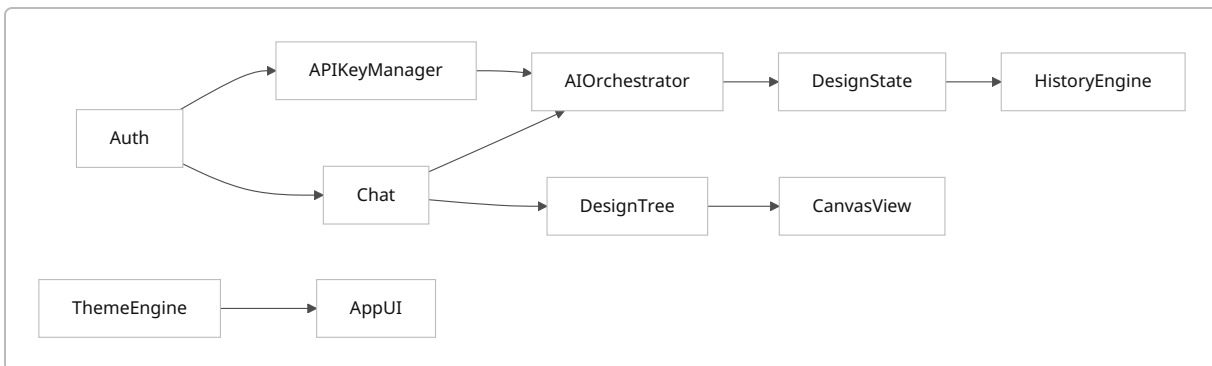
```

/lib
  /auth          # Onboarding (OTP, Google Sign-In)
  /settings      # API key input & key management
  /chat          # Chat UI and Conversation models
  /ai            # Orchestrator, request/response models
  /design         # DesignTree, layers, canvas rendering
  /layers        # UI widgets for each textile section
  /themes        # ThemeData (light, dark, AMOLED)
  /storage       # SecureStorage helpers, cache management
  /utils         # Common utilities (prompt building, validators)

```

This separation keeps features modular.

• **Dependency Graph (Mermaid):** Below is a high-level graph of modules and their dependencies:



(This shows, for example, that the Chat and APIKey modules feed into the AI Orchestrator; the design state connects to the history/undo system; theming drives the overall UI.)

• **Master Fix Prompt for Code Assistant (Z.ai/Claude Code):**

```

### Implement the following fixes in the Flutter codebase:
- Wrap main Scaffold bodies in SafeArea to prevent UI under status bars/
  cutouts [35†L589-L598] .
- Add a UI form in settings: input box for API key with save/remove
  actions. Store the key using flutter_secure_storage (Keychain/Keystore)
  [22†L75-L83] . Validate the key by an API call; show success (green) or
  error (red) state.
- Modify chat input widget: add an "attach file" button that opens the
  system picker (use file_picker). Handle selected files (PNG, JPEG,
  SVG, .febric) and convert to ConversationItem. Show upload progress and
  handle failures [32†L119-L127] .
- Extend ConversationItem model to include `type: 'image'` and relevant
  metadata.
- Expand the DesignTree widget: add sections Dupatta, Kurta, Pant, Border

```

as nodes. Under each, list layers (Top Pallu, Middle Field, etc for Dupatta) with lock and visibility toggles.

- For lock toggle: implement logic so locked layers cannot be modified by AI or UI changes.
- In Theme settings, include theme switch (Light/Dark/AMOLED) and accent color picker. Apply these to ThemeData.
- Ensure all text and controls have proper accessibility labels (ARIA) [3†L113-L117] and high contrast.
- Check performance: avoid using `Opacity` widget for animations; use `FadeInImage` or `AnimatedOpacity` [48†L924-L932]. Do not rebuild entire widget trees unnecessarily.

After implementing, run formatting and linting tools. Verify that landscape mode and all described features (file attach, API key banner, design tree) work correctly.

These recommendations and diagrams are grounded in official Flutter guidance and security best practices [54†L111-L116] [22†L75-L83]. They provide a comprehensive blueprint for implementing the required FEBRIC features with a clean, responsive design and robust architecture.

Sources: Official Flutter docs (SafeArea/MediaQuery [35†L589-L598] [36†L75-L82], performance [48†L900-L905]), Google/AI docs on authentication [17†L276-L284], and best-practice guides on OTP and secure storage [54†L111-L116] [22†L75-L83] [25†L139-L142]. Each recommendation above is supported by these sources or standard UX/security principles.
